

Program No. 01. >> Simple Bar Chart Visualization

Objective: To understand basic data visualization using bar charts.

Tasks:

- Create a simple bar chart using student marks or sales data
- Explain what the bar chart represents

Outcome: Students will be able to visualize and compare categorical data using bar charts.

Tools Used: Python, NumPy, Matplotlib

Dataset: Uncleaned_Student_Dataset.csv

Script/Code:

```
# Objective: Learn creation and manipulation of NumPy arrays
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Load the dataset

```
df = pd.read_csv("Uncleaned_Student_Dataset.csv")
```

Create 1D NumPy array from SGPA column

```
sgpa_array = np.array(df["sgpa"])
print("1D Array (SGPA):")
print(sgpa_array[:10])
```

Create 2D NumPy array from Attendance and Internal Marks

```
two_d_array = np.array(df[["attendance_percentage", "internal_marks"]])
print("\n2D Array (Attendance, Internal Marks):")
print(two_d_array[:5])
```

Indexing and slicing

```
print("\nSliced SGPA Array (Index 2 to 6):")
print(sgpa_array[2:7])
```

```
# Clean SGPA values for numerical operations
```

```
sgpa_clean = pd.to_numeric(df["sgpa"], errors="coerce").dropna()
```

Element-wise operation example

```
print("\nElement-wise Operation (SGPA + 1):")
print((sgpa_clean.values + 1)[:10])
```

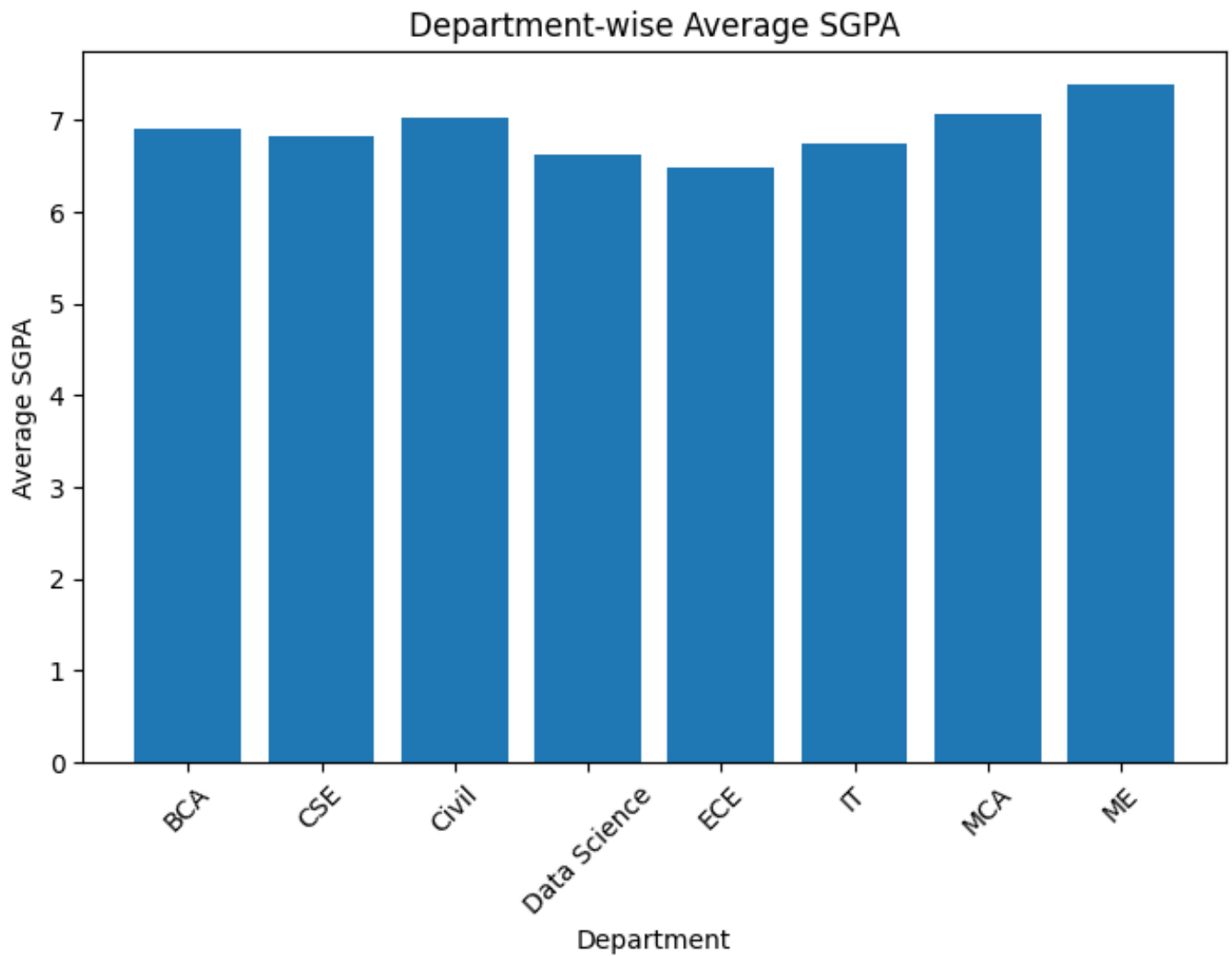
```
# Select relevant data for analysis
```

```
df_bar = df[["department", "sgpa"]].copy()
df_bar["sgpa"] = pd.to_numeric(df_bar["sgpa"], errors="coerce")
df_bar.dropna(inplace=True)
```

Department-wise average SGPA

```
dept_avg_sgpa = df_bar.groupby("department")["sgpa"].mean()
```

```
# Bar chart visualization
plt.figure(figsize=(8, 5))
plt.bar(dept_avg_sgpa.index, dept_avg_sgpa.values)
plt.xlabel("Department")
plt.ylabel("Average SGPA")
plt.title("Department-wise Average SGPA")
plt.xticks(rotation=45)
plt.show()
```

Output:

Thank You!!

Program No. 02. >> Line Chart for Time-Based Data

Objective: To represent trends in time-based data.

Tasks:

- Draw a line chart to represent monthly attendance or sales data
- Interpret trends over time

Outcome: Students will understand trend analysis using line charts.

Tools Used: Python, Matplotlib

Script:

```
# Program No. 02: Line Chart for Time-Based Data
# Objective: Visualize time-based trends using a line chart

import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv("Uncleaned_Student_Dataset.csv")

# Select required columns
df_line = df[["academic_year", "attendance_percentage"]].copy()

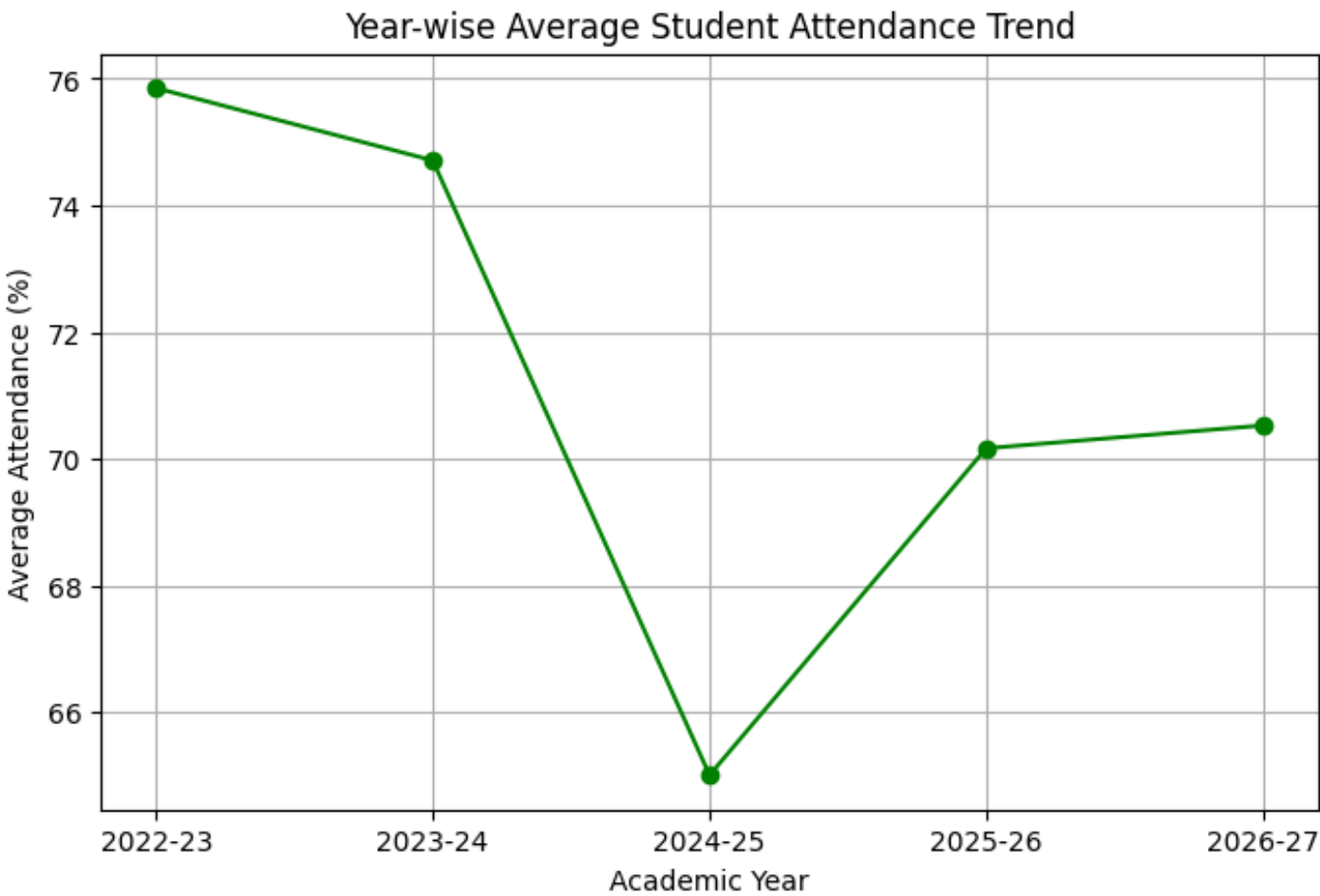
# Convert attendance to numeric and remove missing values
df_line["attendance_percentage"] = pd.to_numeric(
    df_line["attendance_percentage"], errors="coerce"
)
df_line.dropna(inplace=True)

# Calculate year-wise average attendance
yearly_attendance = (
    df_line
    .groupby("academic_year")["attendance_percentage"]
    .mean()
    .reset_index()
    .sort_values("academic_year")
)

# Create the line chart
plt.figure(figsize=(8, 5))
plt.plot(
    yearly_attendance["academic_year"],
    yearly_attendance["attendance_percentage"],
    marker="o"
)
plt.xlabel("Academic Year")
plt.ylabel("Average Attendance (%)")
plt.title("Year-wise Average Student Attendance Trend")
plt.grid(True)
plt.show()
```

Output:

	academic_year	attendance_percentage
0	2022-23	75.850476
1	2023-24	74.708571
2	2024-25	65.007619
3	2025-26	70.165500
4	2026-27	70.525909



Program No. 03. >> Data Representation vs Data Presentation

Objective: To differentiate between data representation and data presentation.

Tasks:

- Represent the same dataset using at least two different chart types
- Compare clarity and interpretation

Outcome: Students will be able to choose appropriate visualization techniques.

Tools Used: Python, Pandas, Matplotlib

Script:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("Uncleaned_Student_Dataset.csv")

# Select required columns
df_rep = df[["department", "sgpa"]].copy()

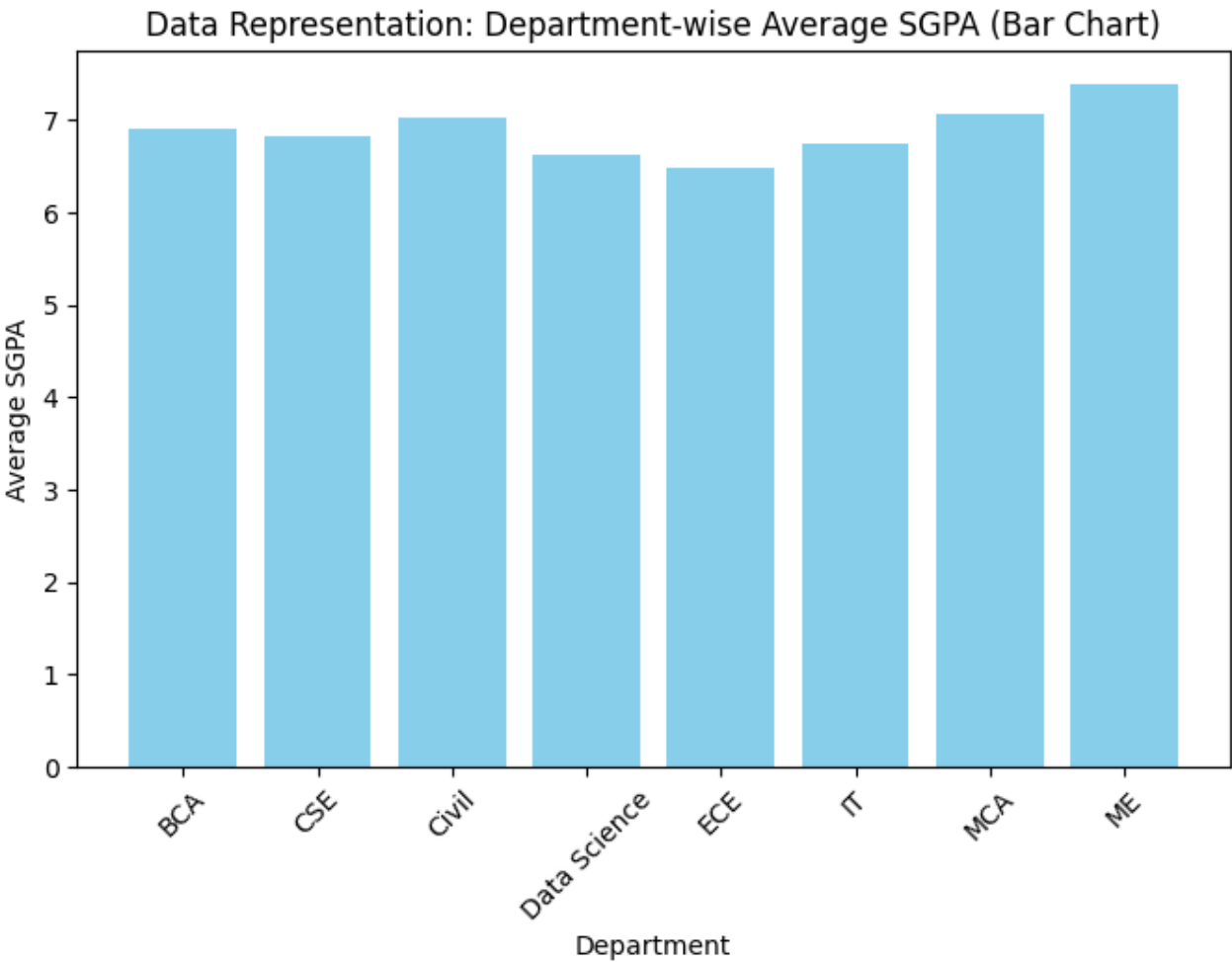
# Convert SGPA to numeric and remove invalid values
df_rep["sgpa"] = pd.to_numeric(df_rep["sgpa"], errors="coerce")
df_rep.dropna(inplace=True)

# Calculate department-wise average SGPA
dept_avg = (
    df_rep
    .groupby("department")["sgpa"]
    .mean()
    .reset_index()
)

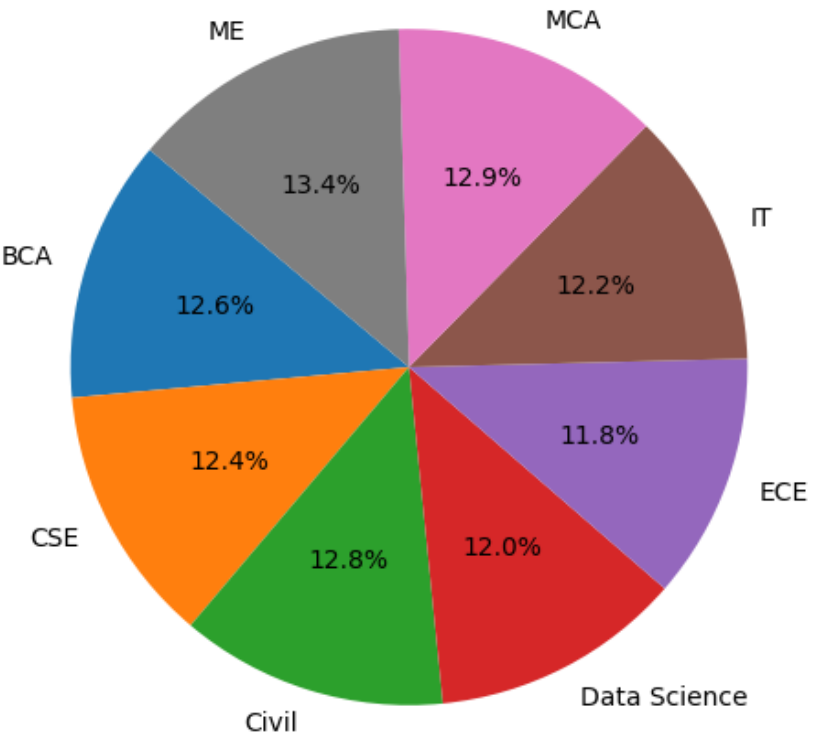
# Data Representation: Bar Chart (Clear Comparison)
plt.figure(figsize=(8, 5))
plt.bar(dept_avg["department"], dept_avg["sgpa"])
plt.xlabel("Department")
plt.ylabel("Average SGPA")
plt.title("Data Representation: Department-wise Average SGPA (Bar Chart)")
plt.xticks(rotation=45)
plt.show()

# Data Presentation: Pie Chart (Visual Distribution)
plt.figure(figsize=(6, 6))
plt.pie(
    dept_avg["sgpa"],
    labels=dept_avg["department"],
    autopct="%1.1f%%",
    startangle=140
)
plt.title("Data Presentation: Department-wise SGPA Distribution (Pie Chart)")
plt.show()
```

Output:



Data Presentation: Department-wise SGPA Distribution (Pie Chart)



Program No. 04. >> Donut Chart and Purpose of Visualization

Objective: To understand the purpose of data visualization.

Tasks:

- Identify visualization goals for a dataset
- Create a donut chart to represent proportional data

Outcome: Students will understand how visualization supports decision-making.

Tools Used: Python, Matplotlib

Script:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv("MMDU_Student_Performance.csv")

# Count students by specialization
specialization_counts = df["specialization"].value_counts()

# Create a donut chart
plt.figure(figsize=(7, 7))
plt.pie(
    specialization_counts,
    labels=specialization_counts.index,
    autopct="%1.1f%%",
    startangle=90,
    wedgeprops={"width": 0.4}
)

plt.title("Distribution of Students by Specialization")
plt.show()
```

Output:

Distribution of Students by Specialization (Donut Chart)

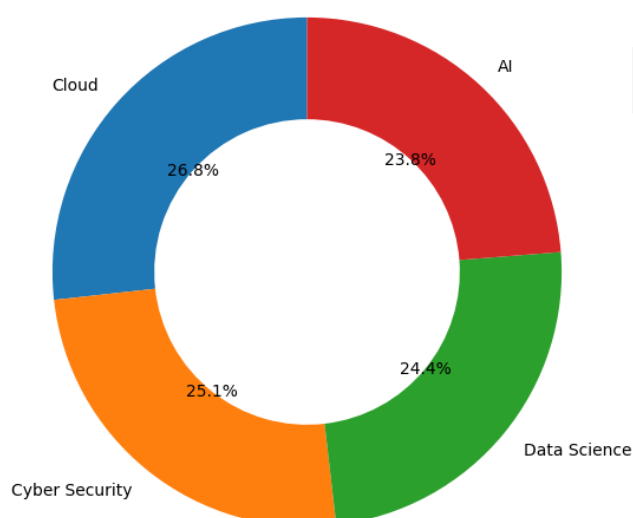
Dataset Used: MMDU_Student_Performance.csv

Load dataset



```
df = pd.read_csv("MMDU_Student_Performance.csv")
```

Python



Program No. 05. >> Seven Stages of Data Visualization

Objective: To study the complete data visualization process.

Tasks:

- Explain the seven stages of data visualization using a simple example

Outcome: Students will understand the end-to-end visualization workflow.

Tools Used: Python, MS Word / Notebook (for explanation)

Script: Uncleaned_Student_Dataset

Attributes include: student_id, roll_number, department, program, academic_year, course_code, attendance_percentage, internal_marks, external_marks, sgpa, gender, age, study_hours

Note: The dataset contains:

- Missing values
- Inconsistent entries (e.g., gender formats, study_hours as text)
- Blank fields: This makes it suitable for explaining the complete visualization workflow.

Seven Stages of Data Visualization

1. Data Collection: Data is collected from raw sources without modification.

Example: Student academic and demographic records collected from different departments and stored as **Uncleaned_Student_Dataset**.

```
df = pd.read_csv('Uncleaned_Student_Dataset.csv')  
  
print(df.head())  
  
print(df.info())
```

2. Data Cleaning: Incorrect, missing, and inconsistent data is identified and handled.

Example:

- Removing or filling missing sgpa values
- Converting gender values like M, Male, female into a standard format
- Replacing text values like five in study_hours with numeric values

```
# Handle missing values in sgpa  
df['sgpa'] = df['sgpa'].fillna(df['sgpa'].mean())  
df['course_code'] = df['course_code'].fillna('Unknown')  
df['internal_marks'] = df['internal_marks'].fillna(df['internal_marks'].mean())  
df['academic_year'] = df['academic_year'].fillna(df['academic_year'].mode())  
df['age'] = df['age'].fillna(df['age'].mean())  
df['study_hours'] = df['study_hours'].fillna(df['study_hours'].mode())  
df['external_marks'] = df['external_marks'].fillna(df['external_marks'].mean())  
df['attendance_percentage'] =  
df['attendance_percentage'].fillna(df['attendance_percentage'].mean())  
  
# Standardize gender values  
df['gender'] = df['gender'].str.upper().replace({'FEMALE': 'F', 'MALE': 'M'})
```

```
# Convert text study_hours to numeric
study_hours_mapping = {'zero': 0, 'one': 1, 'two': 2, 'three': 3, 'four': 4,
                        'five': 5, 'six': 6, 'seven': 7}
df['study_hours'] = pd.to_numeric(df['study_hours'], errors='coerce')
df['study_hours'] = df['study_hours'].fillna(df['study_hours'].mean())

print(df.head())
print(df.info())
```

3. Data Processing: Cleaned data is organized and transformed into usable form.

Example:

- Calculating total marks = internal + external
- Categorizing attendance as low, medium, or high
- Creating age groups

```
# Calculate total marks
df['total_marks'] = df['internal_marks'] + df['external_marks']

# Categorize attendance as low, medium, or high
df['attendance_category'] = pd.cut(df['attendance_percentage'],
                                   bins=[0, 50, 75, 100],
                                   labels=['Low', 'Medium', 'High'])

# Create age groups
df['age_group'] = pd.cut(df['age'],
                        bins=[0, 20, 25, 30, 100],
                        labels=['<20', '20-25', '25-30', '30+'])

print(df[['student_id', 'internal_marks', 'external_marks', 'total_marks',
          'attendance_percentage', 'attendance_category', 'age', 'age_group']].head(10))
print(df.info())
```

4. Data Analysis: Patterns and relationships are explored in the processed data.

Example:

- Analyzing relationship between study_hours and sgpa
- Comparing performance across departments
- Identifying students with low attendance and low marks

```
# Analyzing relationship between study_hours and sgpa
correlation = df['study_hours'].corr(df['sgpa'])
print(f'Correlation between study_hours and sgpa: {correlation:.4f}')

# Comparing performance across departments
dept_performance = df.groupby('department')[['sgpa', 'total_marks',
        'attendance_percentage']].mean()
```

```
print("\nDepartment-wise Performance:")
print(dept_performance)

# Identifying students with low attendance and low marks
low_attendance_marks =
df[(df['attendance_percentage'] < 50) & (df['total_marks'] < 50)]
print(f"\nStudents with low attendance (<50%) and low marks (<50):
{len(low_attendance_marks)}")
print(low_attendance_marks[['student_id', 'department', 'attendance_percentage',
'total_marks', 'sgpa']])
```

5. Data Visualization: Graphs and charts are created to represent the analyzed data.

Example:

- Bar chart for department-wise student count
- Pie or donut chart for gender distribution
- Line chart showing academic year trends

```
# Bar chart for department-wise student count
dept_count = df['department'].value_counts()
plt.figure(figsize=(10, 5))
dept_count.plot(kind='bar', color='skyblue')
plt.title('Department-wise Student Count')
plt.xlabel('Department')
plt.ylabel('Number of Students')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

6. Interpretation: The visual outputs are examined to derive meaningful insights.

Example:

- Higher study hours generally lead to better SGPA
- Some departments have consistently higher attendance
- Certain programs need academic support

```
# Section 6: Interpretation - Key Insights

print("=" * 60)
print("KEY INSIGHTS FROM DATA VISUALIZATION")
print("=" * 60)

# Insight 1: Study Hours vs SGPA Correlation
print(f"\n1. Study Hours vs SGPA Correlation: {correlation:.4f}")
print(" → Weak negative correlation suggests study hours alone don't determine SGPA")
print(" → Other factors like course difficulty and teaching quality matter more")
```

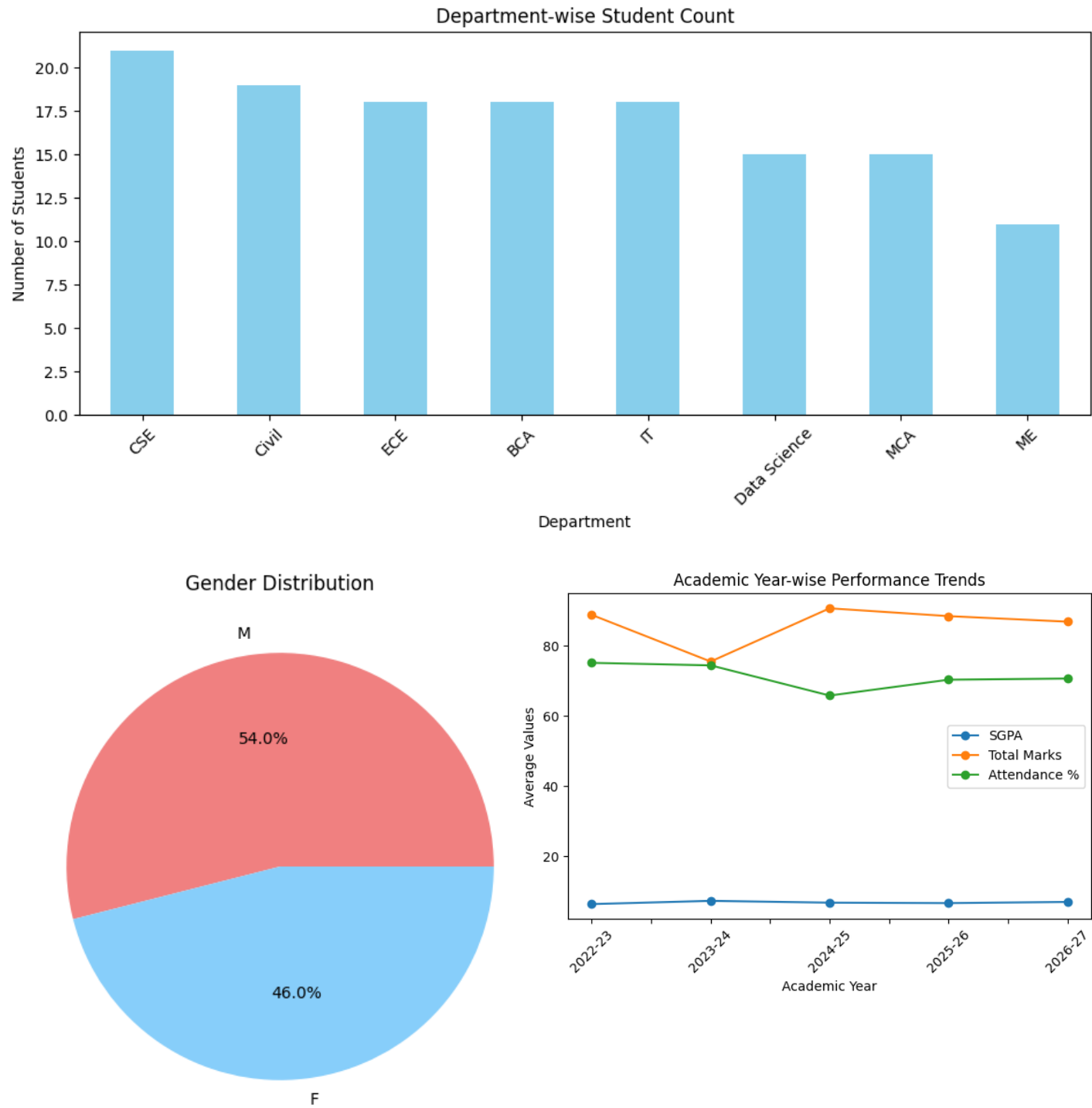
7. Decision Making: Final decisions are taken based on interpreted insights.

Example:

- Planning remedial classes
- Improving attendance policies
- Allocating faculty based on department strength

Outcome: Students understand the end-to-end data visualization workflow, from an uncleaned dataset to insight-driven decision making.

Output:



Program No. 06. >> Basic Visualization Using Any Tool

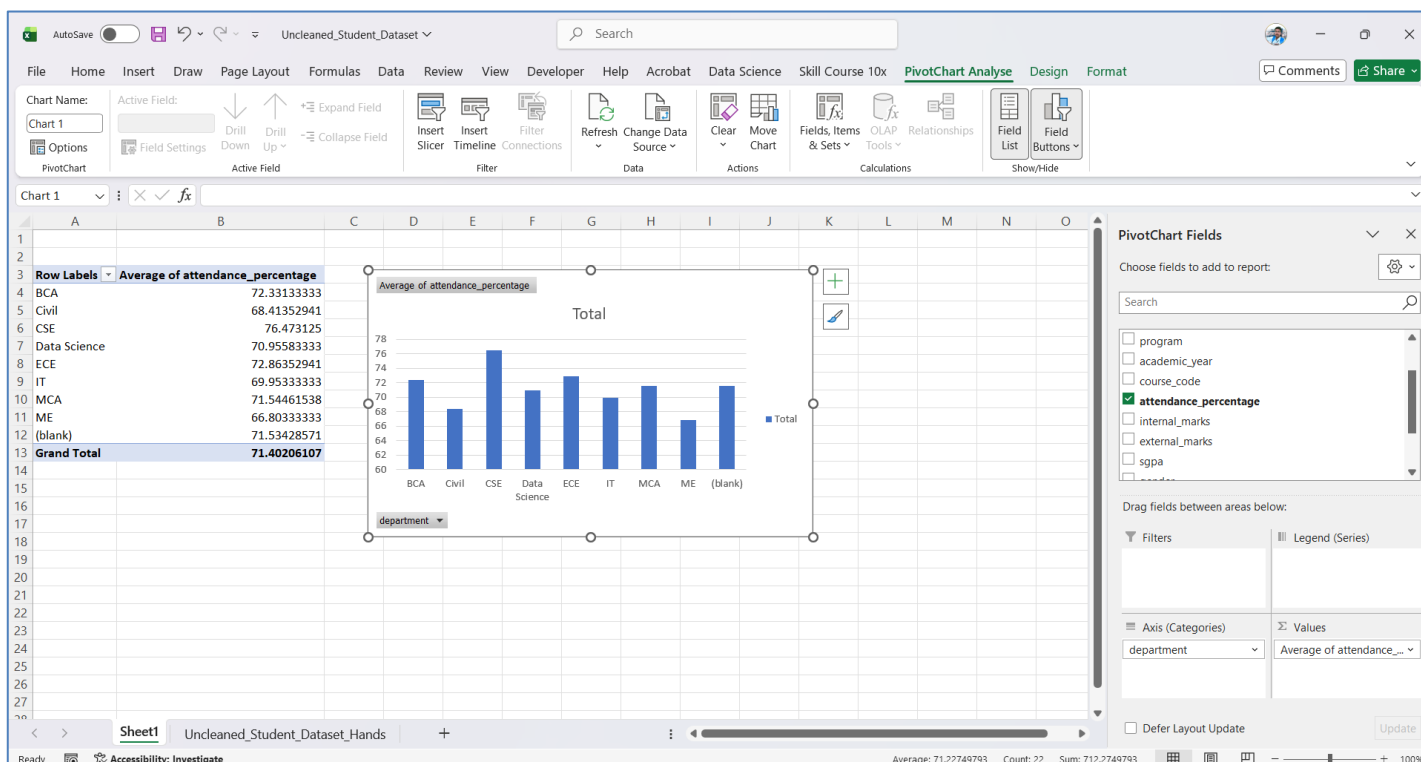
Objective: To gain hands-on experience with visualization tools.

Tasks:

- Create one basic visualization using Excel / Google Sheets / Tableau / Power BI

Outcome: Students will be able to create basic charts using visualization tools.**Tools Used:** Excel / Google Sheets / Tableau / Power BI / Python**Script:****#Bar Chart – Department-wise Average Attendance****→ Steps to Create Visualization in Excel**

1. Open **Microsoft Excel**
2. Import the **Uncleaned_Student_Dataset.xlsx**
3. Make formatting:
 - Fit Column and Width: **Alt + H + O + I**, **Alt + H + O + A**
 - Formatting Data: Select row1 '**Ctrl + B**', **Alt + H + B + A**
4. Select the columns:
 - department
 - attendance percentage
5. Use **Insert → Pivot Table**
 - Rows: Department
 - Values: Average of Attendance Percentage
6. Select the Pivot Table
7. Go to **Insert → Bar Chart**
8. Add:
 - Chart Title
 - X-axis: Department
 - Y-axis: Average Attendance (%)

Output:

Program No. 07. >> Area Chart for Time-Series Data

Objective: To visualize cumulative data over time.

Tasks:

- Create an area chart for yearly profit or rainfall data

Outcome: Students will understand growth patterns using area charts.

Tools Used: Python, Matplotlib

Script:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("MMDU_Student_Performance.csv")

# Select required columns
data = df[['academic_year', 'percentage']]

# Remove missing values
data = data.dropna()

# Convert academic_year to string (for plotting)
data['academic_year'] = data['academic_year'].astype(str)

# Calculate average percentage per academic year
yearly_avg = data.groupby('academic_year')['percentage'].mean().sort_index()

# Create area chart
plt.figure(figsize=(8, 5))
plt.fill_between(
    yearly_avg.index,
    yearly_avg.values,
    alpha=0.7
)

# Labels and title
plt.xlabel("Academic Year")
plt.ylabel("Average Percentage")
plt.title("Area Chart Showing Average Student Performance Over Academic Years")

# Display chart
plt.show()
```

Output:

Program No. 08. >> Scatter Plot Analysis

Objective: To study relationships between two variables.

Tasks:

- Draw a scatter plot for study hours vs marks
- Analyze correlation

Outcome: Students will identify relationships between variables.

Tools Used: Python, Matplotlib

Script:

Output:

Program No. 09. >> Pivot Table and Pivot Chart

Objective: To summarize and analyze data.

Tasks:

- Create a pivot table
- Generate a pivot chart from the dataset

Outcome: Students will perform grouped data analysis efficiently.

Tools Used: Python, Pandas, Matplotlib / Excel

Script:

Output:

Program No. 10. >> Tree Map Visualization

Objective: To visualize hierarchical and categorical data.

Tasks:

- Design a tree map for department-wise student count

Outcome: Students will understand hierarchical data visualization.

Tools Used: Python, Squarify, Matplotlib

Script:**Output:**

Program No. 11. >> Node-Link Diagram

Objective: To visualize relationships between entities.

Tasks:

- Draw a node-link diagram showing relationships (e.g., friends in a class)

Outcome: Students can build and train basic neural networks.

Tools Used: Python, NetworkX, Matplotlib

Script:**Output:**

Program No. 12. >> Heatmap for Correlation Analysis

Objective: To analyze correlation among numerical variables.

Tasks:

- Create a color-coded correlation matrix (heatmap)

Outcome: Students will analyze relationships among multiple variables.

Tools Used: Python, Pandas, Seaborn, Matplotlib

Script:

Output:

Practical 13: >> Multi-Variable Time-Series Visualization

Objective: To analyze multiple variables over time.

Tasks:

- Create a time-series visualization with multiple variables
- Analyze observed patterns

Outcome: Students will interpret multi-dimensional time-series data.

Tools Used: Python, Matplotlib

Script:**Output:**

Thank You!

Program No. 14. >> Text Data Visualization

Objective: To visualize textual data.

Tasks:

- Count word frequency from a paragraph
- Plot text visualization

Outcome: Students will understand basic text visualization techniques.

Tools Used: Python, WordCloud, Matplotlib

Script:

Output:

Program No. 15. >> Multivariate Chart Visualization

Objective: To visualize more than two variables simultaneously.

Tasks:

- Create a multivariate chart using marks in three subjects

Outcome: Students will analyze complex datasets visually.

Tools Used: Python, Matplotlib

Script:

Output:

Program No. 16. >> Case Study on Visualization

Objective: To study real-life applications of visualization.

Tasks:

- Analyze a small case where visualization improves understanding

Outcome: Students will appreciate visualization in real-world decision-making.

Tools Used: Python, Jupyter Notebook / MS Word.

Script:**Output:**

Program No. 17. >> Live-like Data Visualization

Objective: To simulate real-time data visualization.

Tasks:

- Create a live-like visualization using continuously updated values

Outcome: Students will understand real-time monitoring concepts.

Tools Used: Python, Matplotlib, Time module.

Script:**Output:**

Program No. 18. >> Visualization Best Practices

Objective: To learn principles of effective visualization.

Tasks:

- Write rules or best practices for good visualizations with examples

Outcome: Students will design meaningful and clear visualizations.

Tools Used: MS Word / Notebook, Python (examples).

Script:**Output:**

Program No. 19. >> Open and Closed Port Visualization

Objective: To visualize basic network security data.

Tasks:

- Create a chart showing open and closed ports

Outcome: Students will understand visualization of network status.

Tools Used: Python, Matplotlib.

Script:**Output:**

Program No. 20. >> Firewall Log Visualization
Objective: To identify suspicious traffic using visualization.
Tasks:

- Visualize firewall log data
- Identify suspicious traffic patterns

Outcome: Students will detect anomalies using visual analysis.
Tools Used: Python, Pandas, Matplotlib.

Script:

Output:

Program No. 21. >> Intrusion Detection Visualization

Objective: To analyze intrusion detection logs.

Tasks:

- Design intrusion detection log visualization
- Highlight possible attacks

Outcome: Students will understand visualization in cybersecurity.

Tools Used: Python, Pandas, Matplotlib.

Script:

Output:

Program No. 22. >> Sentiment Analysis Visualization

Objective: To analyze sentiment from social media data.

Tasks:

- Implement sentiment analysis on live tweet data using Python libraries

Outcome: Students will apply NLP techniques for visualization.

Tools Used: Python, TextBlob / NLTK, Matplotlib.

Script:**Output:**

Program No. 23. >> Secure Data Visualization Architecture

Objective: To design a secure visualization system.

Tasks:

- Design secured data visualization architecture
- Explain security measures used

Outcome: Students will understand security considerations in visualization systems.

Tools Used: Scikit-learn, Pandas, NumPy.

Script:

Output:

Program No. 24. >> Tree Map Visualization

Objective: To visualize hierarchical and categorical data.

Tasks:

- Design a tree map for department-wise student count

Outcome: Students will understand hierarchical data visualization.

Tools Used: Python (conceptual), Diagram Tool (Draw.io / MS PowerPoint).

Script:**Output:**